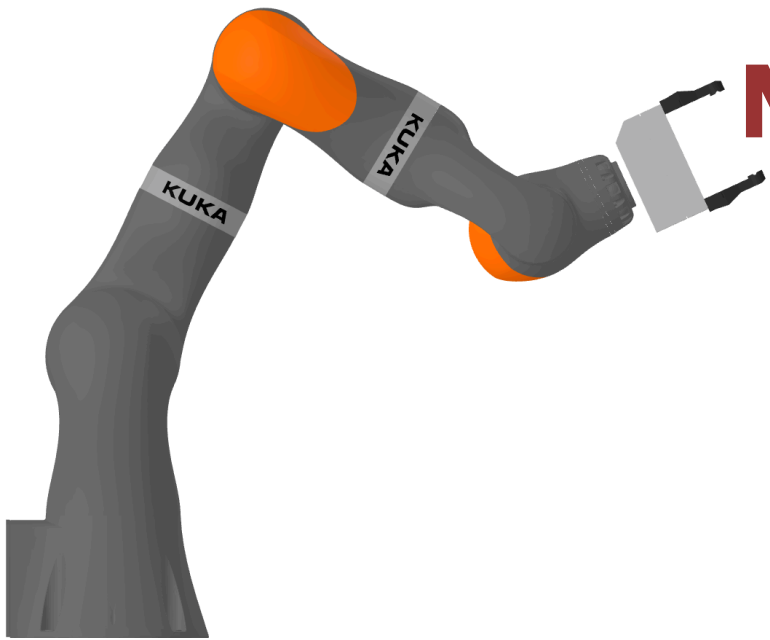
Introduction: Anatomy of a Manipulation System

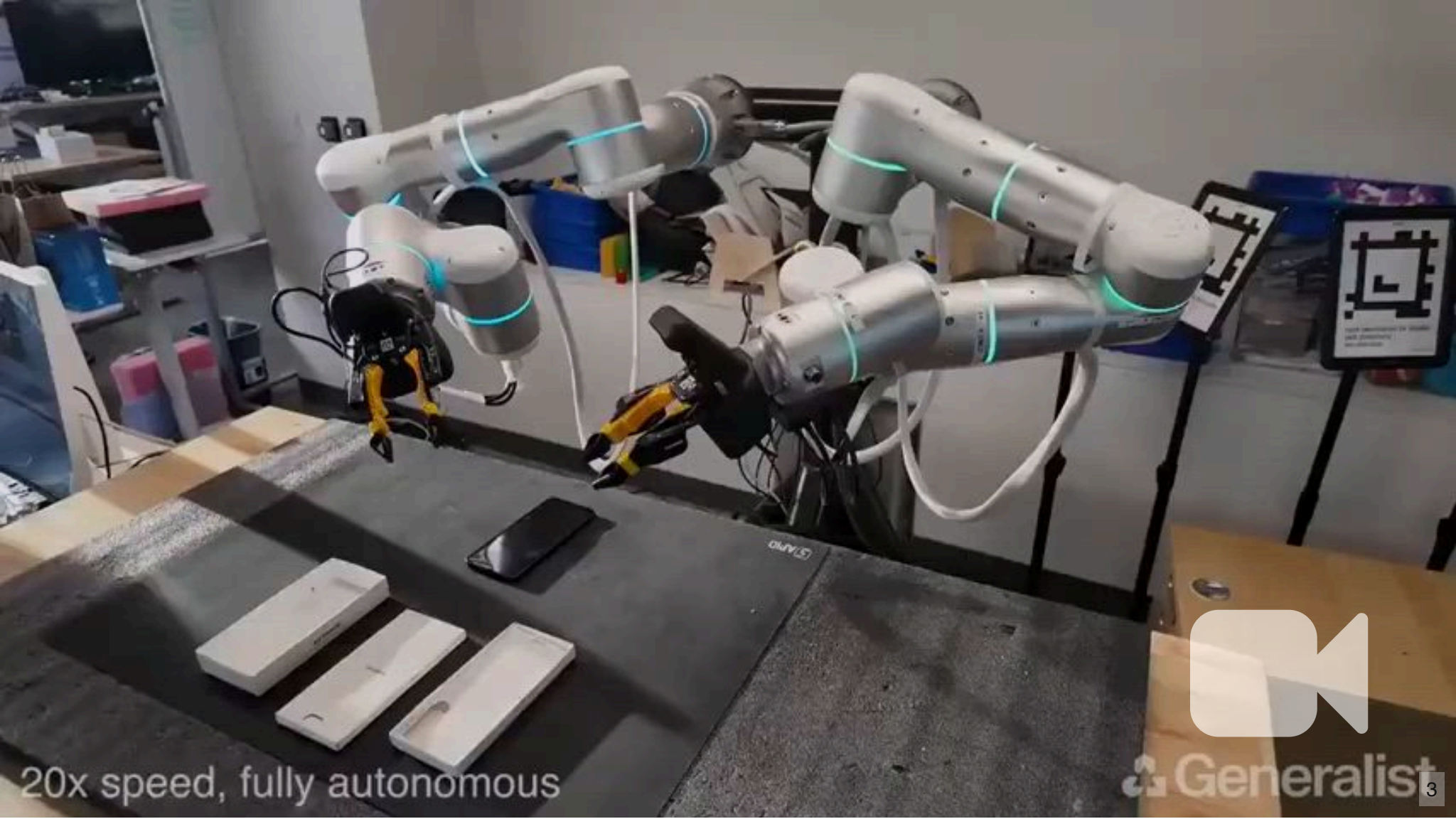


MIT 6.4210/2: Robotic Manipulation

Fall 2026, Lecture 1

Massachusetts Institute of Technology

https://www.sunday.ai/


20x speed, fully autonomous
Generalist

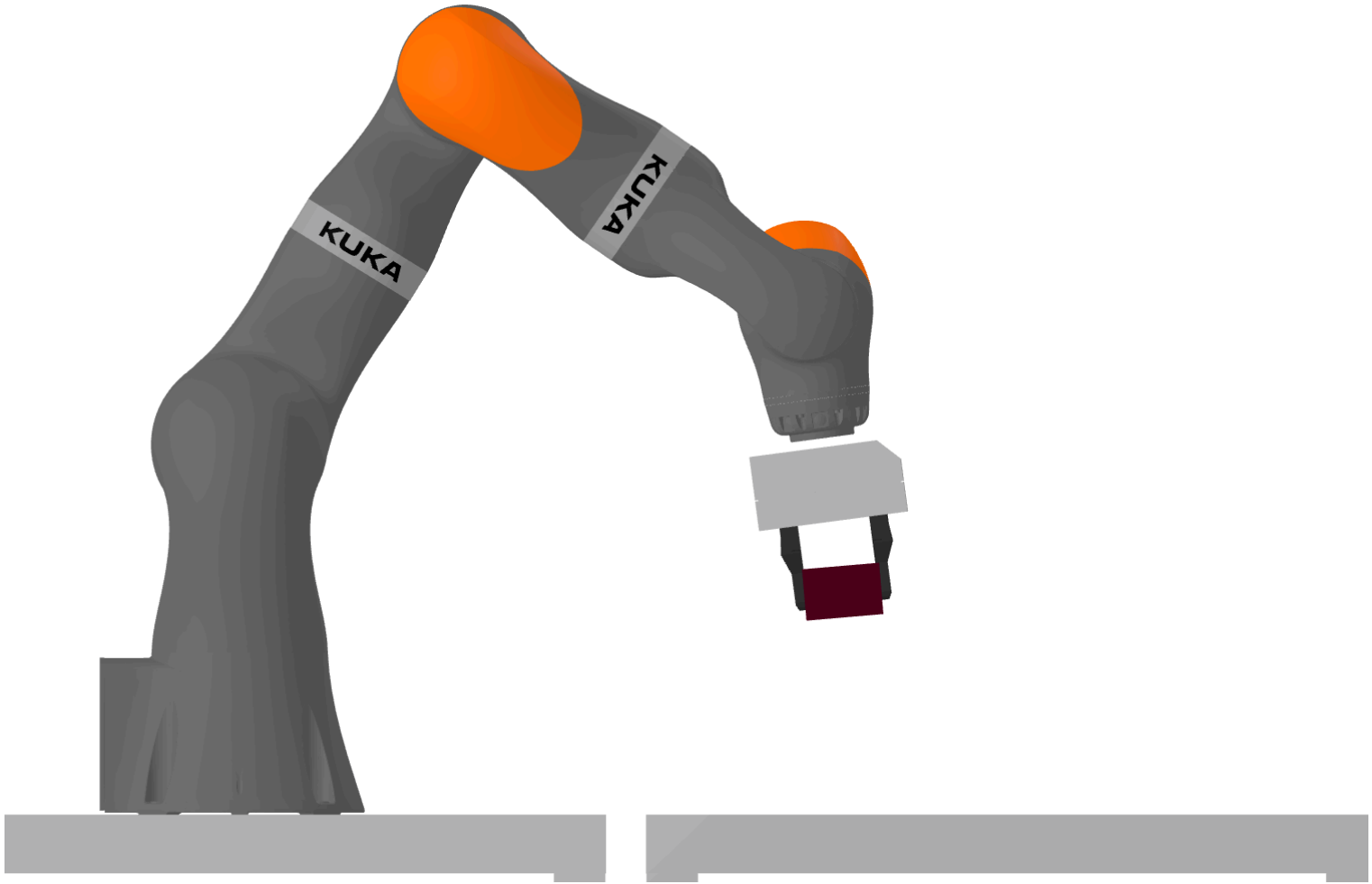
Today

- What do we mean by "***manipulation***"?
- Logistics
- Anatomy of a manipulation system
- Drake

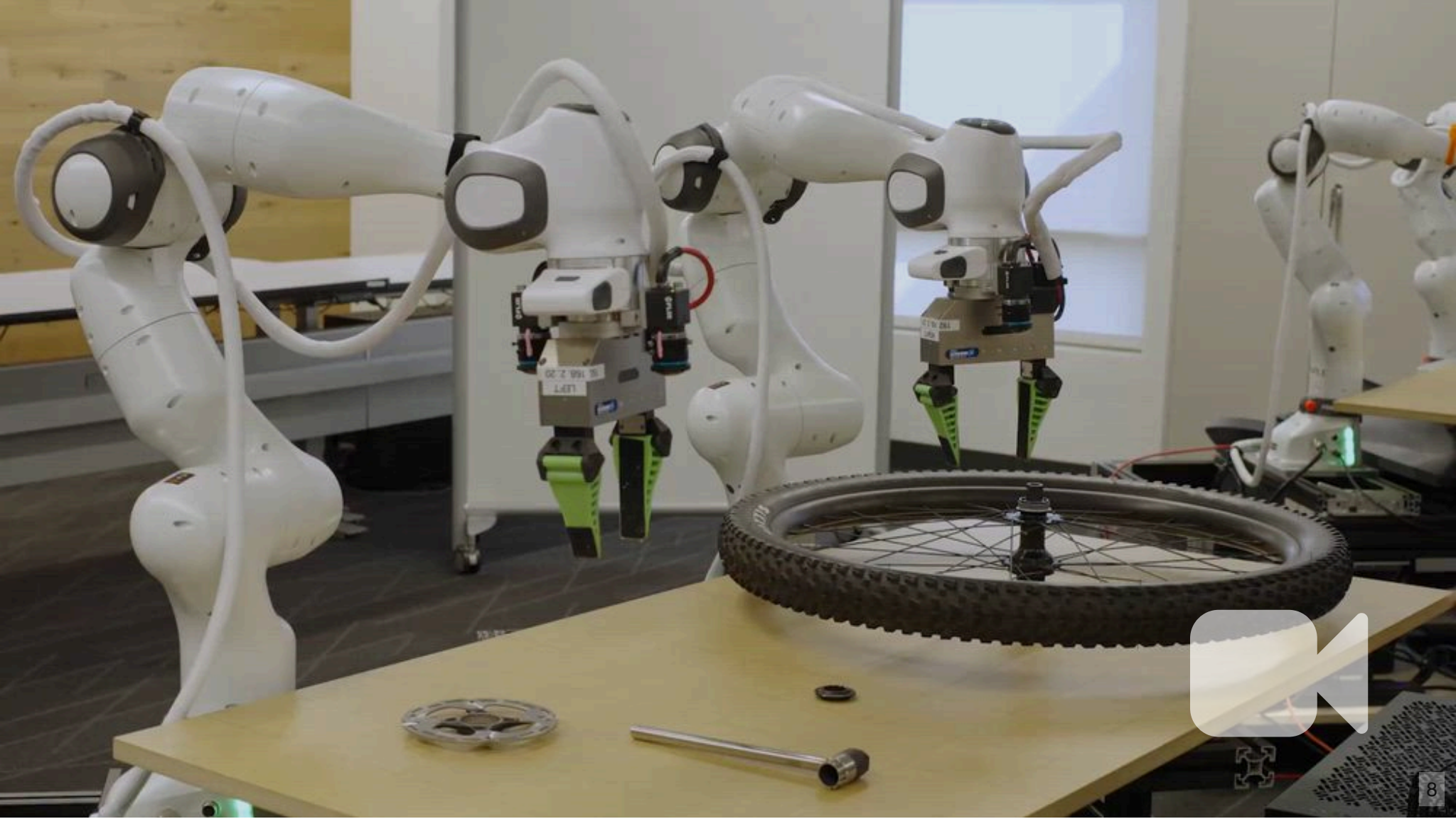
Matt Mason says

- *Definition 1 (etymological).* Manipulation refers to the activities performed by hands.
- ...
- *Definition 5.* Manipulation refers to an agent's control of its environment through selective contact.

Matthew T. Mason. Toward Robotic Manipulation. *Annual Review of Control, Robotics, and Autonomous Systems*, 1:1-28, 2018.


KUKA
KUKA

https://www.youtube.com/embed/1RbaIo4VdbA? enablejsapi=1&start=112&mute=1



Autonomous "Open-world" Manipulation

- *Definition 5.* Manipulation refers to an agent's control of its environment through selective contact.

- *Even broader:* "Open-world" manipulation requires:
  - rich perceptual understanding of the environment,
  - "common-sense" understanding of objects,
  - the ability to make long-term (task-level) plans, and combine them with fine (joint-level) motions.

Course Info

https://manipulation.csail.mit.edu/Fall2026/

Website: http://manipulation.mit.edu/Fall2026

Course Info

https://manipulation.csail.mit.edu/Fall2026/schedule.html

Website: http://manipulation.mit.edu/Fall2026

6.4210 is a CI-M (Communication Intensive in the Major)

Communications Instructors from Comparative Media Studies/Writing

Recitations Fridays (starting Sept 18)

Project-related (CI) assignments for 6.4210 students

1. Journal club presentation *(3 pts)*
2. Project preproposal *(1 pt)*
3. Project proposal final revision *(4 pts)*
4. Project final report *(10 pts)*
5. Project final video *(3 pts)*
6. Peer reviews *(2 pts)*
7. Writing conferences *(2 pt)*
8. CI-M participation & pre-writing tasks *(3 pts)*

Use of AI

- For communication tasks
- Read the full policy

Acceptable	Acceptable with attribution	Not acceptable
GenAI Ideation assistance	GenAI Writing Assistance	GenAI Generated Content
- Use of GenAI to explore possible topics for the design project. - Use of an AI assistant for suggestions on how to make improvements to your writing and oral presentations.	- Use of GenAI to search for and summarize related work. You may use GenAI-generated summaries of related papers with attribution. When you use GenAI to review and summarize related work, make sure to check the generated summary for accuracy against the abstract, introduction and conclusion of the paper in question. - Use of GenAI to proofread and revise your writing. Editorial assistance from GenAI must be disclosed in *Acknowledgements*. - Acknowledged use of GenAI as a process tool in the design of figures, tables, captions and visual elements in the project video. You are responsible for what your visuals communicate: any misrepresentation of results that originates with an AI tool will be your responsibility.	- Use of GenAI to generate any portion of your text. - Use of AI to generate the sections that characterize your approach and report and interpret your experimental results.

Use of AI

- For homework and project implementation- unlimited
- But we expect you to understand what is generated. Exam questions based on material.
- Read the policy

6.4210 vs 6.4212

- Higher expectations on 6.4212 projects
- 6.4212 has extra problems on psets and quizzes

Logistics I

- Make sure you're on Piazza and Gradescope
- CI-M (6.4210) material on Canvas.
- CI-M (6.4210) recitation signup on Canvas (ignore Registrar)
- Recorded lectures via Canvas

- Read the lecture notes
- ~Weekly problem sets
- Midterm quizzes in class 10/14, 11/16 - [9-11am]
- Final project (groups of 3):
  - Group Report
  - Group Video and
  - Individual Summary (in-class questions - 12/9)

Logistics II - Problem sets

- Problem sets out on Thursday, due following Thursday
- You can find link to Pset repo on Schedules page
- Preferably install software on your machine. Using Colab is also supported.
- Solutions are submitted via Gradescope (separate Gradescope for grad/undergrad)

Class Time and Location

Fall semester (September - December, 2026)
Lectures: Monday, Wednesday 9:30 -11:00 am in 45-230
Recitations are only for the undergrad version (6.4210):

- R1: Friday at 12-1pm in TBD,
- R2: Friday at 1-2pm in TBD,
- R3: Friday at 1-2pm in TBD.
- R4: Friday at 1-2pm in TBD.
- R5: Friday at 2-3pm in TBD.
- R6: Friday at 2-3pm in TBD.

Class Attendance

Attendance for everyone is mandatory on:

- October 14 (Quiz 1),
- November 16 (Quiz 2), and
- December 9 (Project summary)

Attendance for 6.4210 students at Friday CI-M recitations is mandatory. Most sessions are in-class activities and brief student presentations.

Office Hours

See Piazza posts for our office hours schedule.

Class Forums

Forums are on Piazza (sign up with your @mit.edu email address).

For 6.4210, communication on CI-M material will primarily happen through Canvas.

We will use Gradescope for problem set submission and quiz grading.

Grading Policy

Late assignments will be penalized 10% every 24 hours. Additionally, we will grant a one time (one pset) extension of up to one week without penalty. This will automatically be applied at the end of the term to whichever pset will earn you the most points; there is no need to request it. All psets will be be assigned equal weight when determining final grade.

6.4210 grade distribution
Assignments: 5%
Midterms: 35%
Journal club/peer reviews: 5%
CI-M participation, pre-writing tasks & writing conferences: 5%
Project proposal: 10% (5% tech grade + 5% CI grade)
Project report and summary: 30% (20% tech grade + 10% CI grade)
Project video: 10% (7% tech grade + 3% CI grade)

6.4212 grade distribution
Assignments: 5%
Midterms: 45%
Project proposal: 5%
Project updates: 10%
Project report, video and summary: 35%

Policy on Generative AI:
For communication tasks: Download the policy (PDF)
For homework and project: Download the policy (PDF)

https://manipulation.csail.mit.edu/intro.html

Autonomous "Open-world" Manipulation

- *Definition 5.* Manipulation refers to an agent's control of its environment through selective contact.

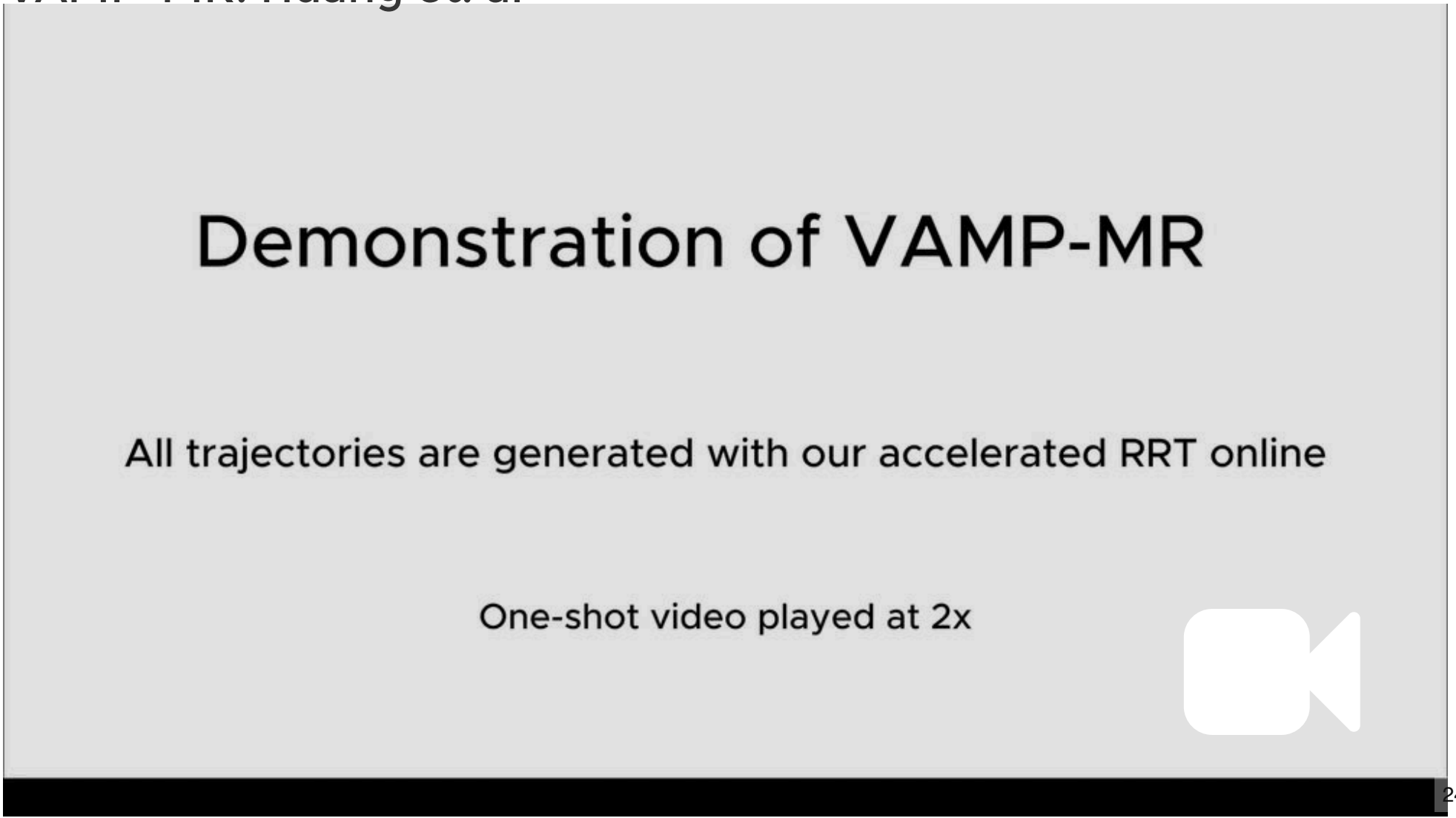
- *Even broader:* "Open-world" manipulation requires:
  - rich perceptual understanding of the environment,
  - "common-sense" understanding of objects,
  - the ability to make long-term (task-level) plans, and combine them with fine (joint-level) motions.

https://www.youtube.com/embed/F_7IPm7f1vI?enablejsapi=1&mute=1

Dexterity vs Generality

https://www.youtube.com/embed/hxsWeVtb-JQ?enablejsapi=1

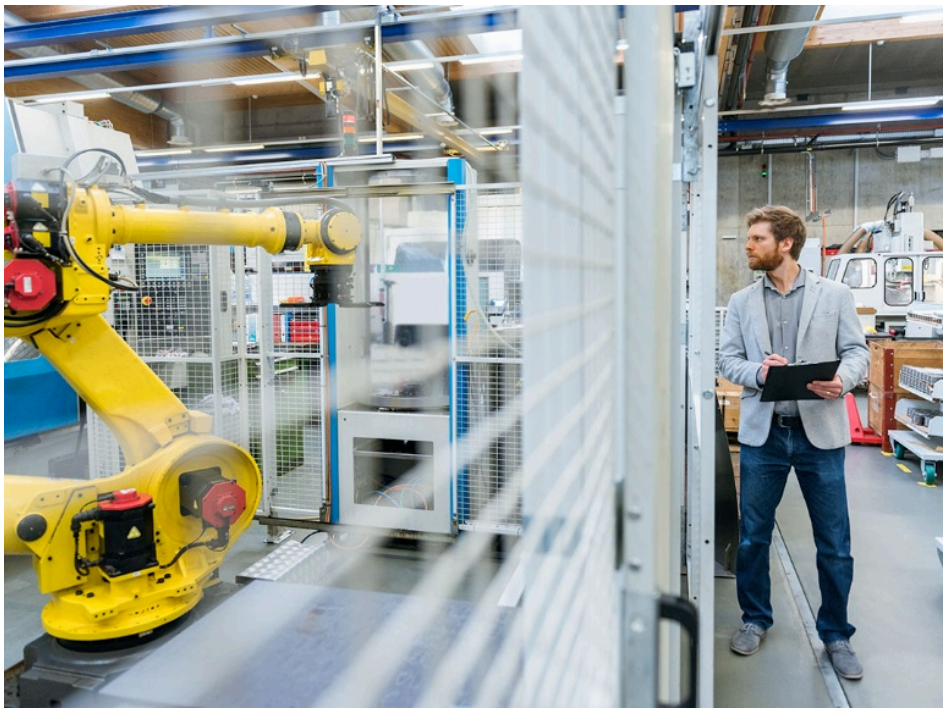
VAMP-MR: Huang et. al



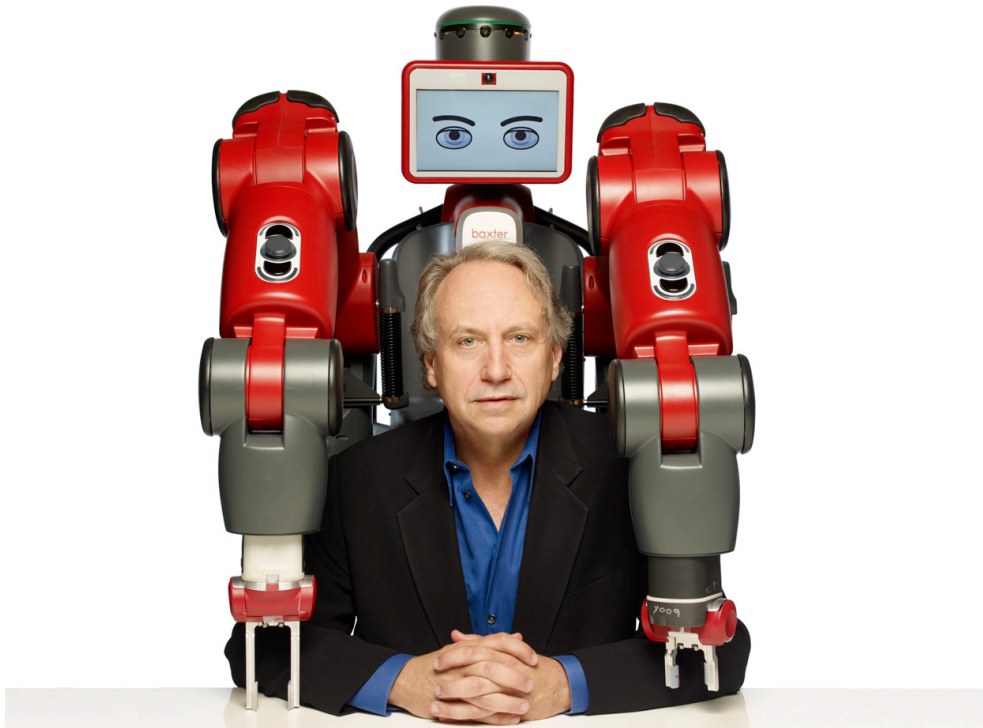
Basic components

- Links
- Joints (Revolute, Prismatic)
- Sensors
  - Proprioception (position, torque)
  - Force and/or tactile
  - Vision: RGB(D), Laser Scan
- Actuation
  - Motors (electric, hydraulic)
  - Transmission (gear reduction, backlash, friction)
  - Motor drivers (position control, torque control)

Exploring robot arms

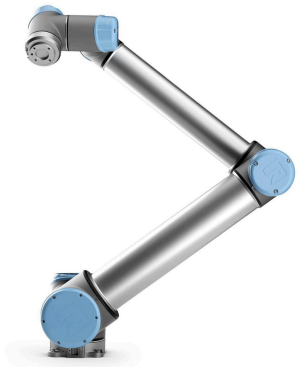


https://www.infineon.com/cms/en/discoveries/robotics-systems-safety-security/



https://spectrum.ieee.org/robotics/industrial-robots/rethink-robotics-baxter-robot-factory-worker

Universal Robots



Flexiv



Kuka iiwa





Kinova Jaco



ABB Yumi



Franka Panda

https://www.youtube.com/embed/dnUwqngH0bM?start=35&mute=1&enablejsapi=1

https://www.youtube.com/embed/P-d-yTcxdzw?enablejsapi=1

Schunk WSG 50



Force-controlled planar gripper

Hands



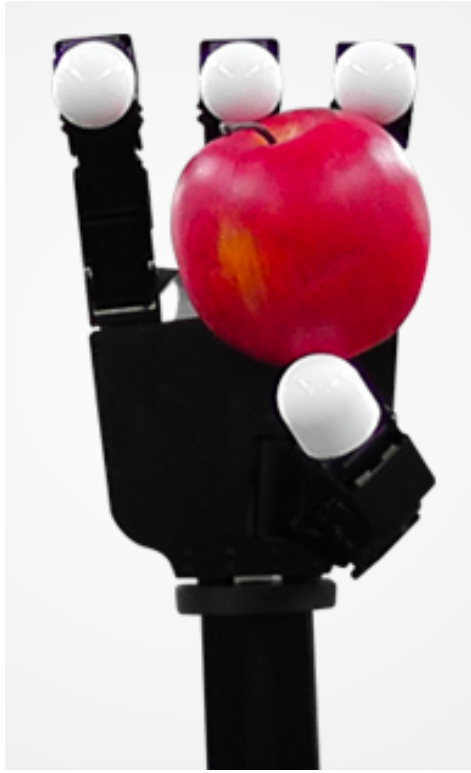
Shadow hand



iHY (i-Robot, Harvard, Yale)

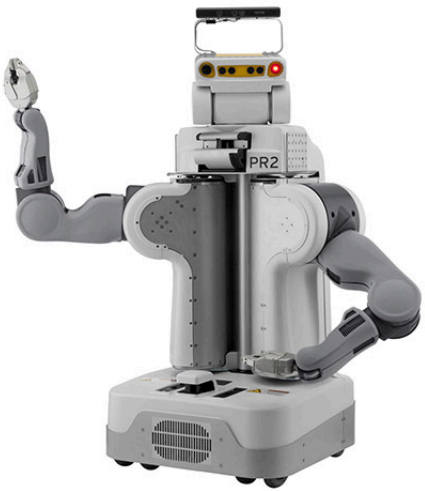


Robotiq 3-fingered gripper



Allegro Hand

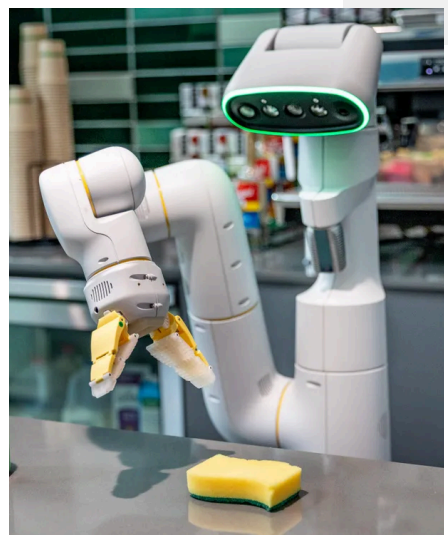
Mobile Manipulators











Mobile manipulators

https://www.youtube.com/embed/I3_GIb7W5UQ?enablejsapi=1&mute=1

Rainbow RBY1

Humanoids (w/ dexterous hands)

https://www.youtube.com/embed/0SRVJaOg9Co?enablejsapi=1&mute=1

Figure

This class

- Robot ***arms***.
  - The hardware matters.
  - We'll often use *Kuka iiwa*
- Robot ***hands***.
  - We'll often use *Schunk WSG*

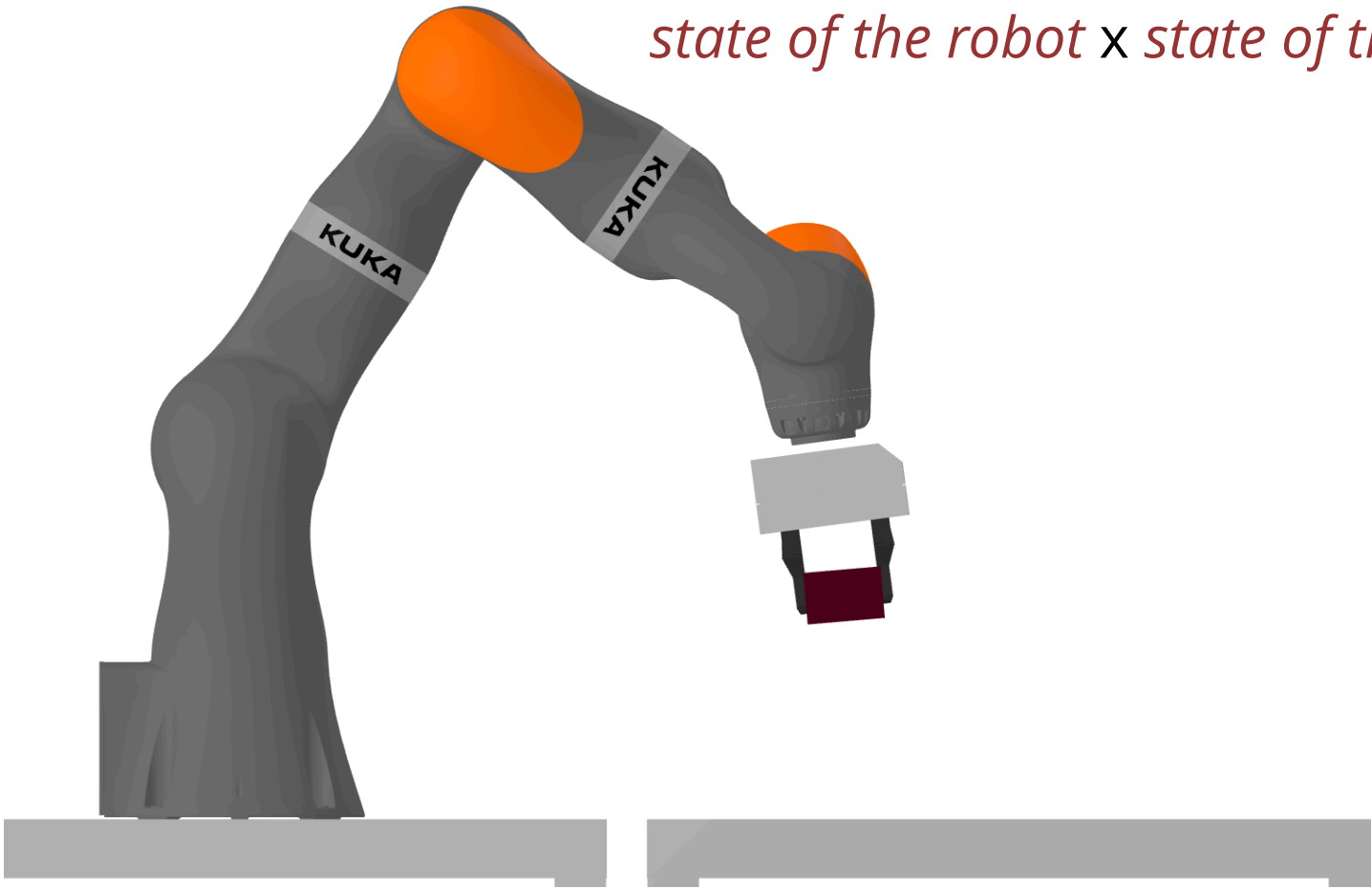


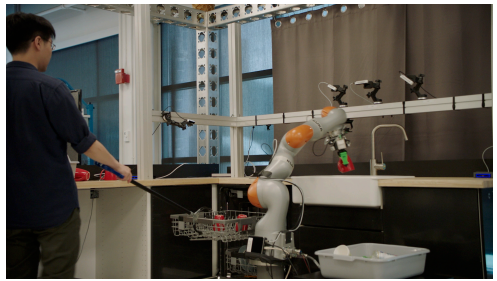


The *anatomy* of a manipulation system

To be clear: we're not just controlling the arm

state of the robot x *state of the environment*

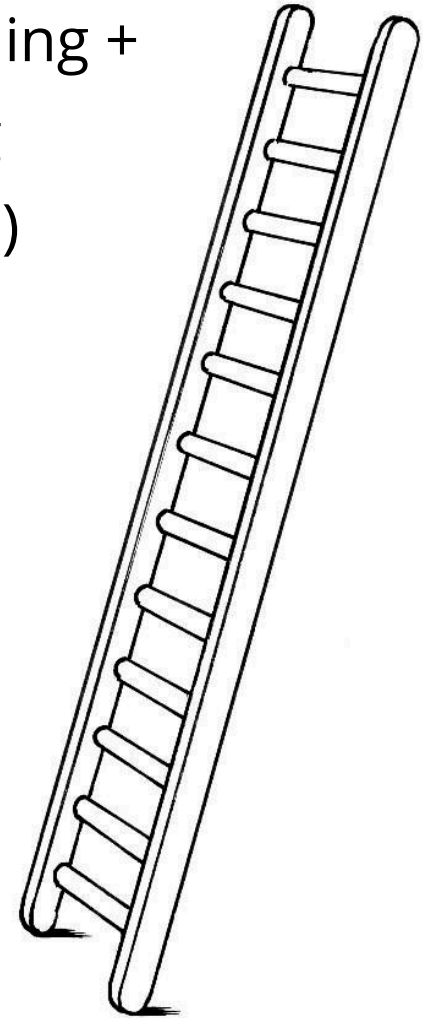


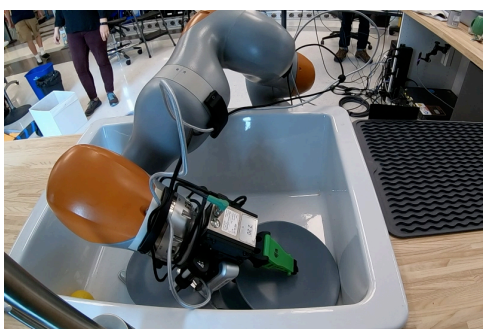


Scene-understanding +

Task-level Planning

(discrete/symbolic)

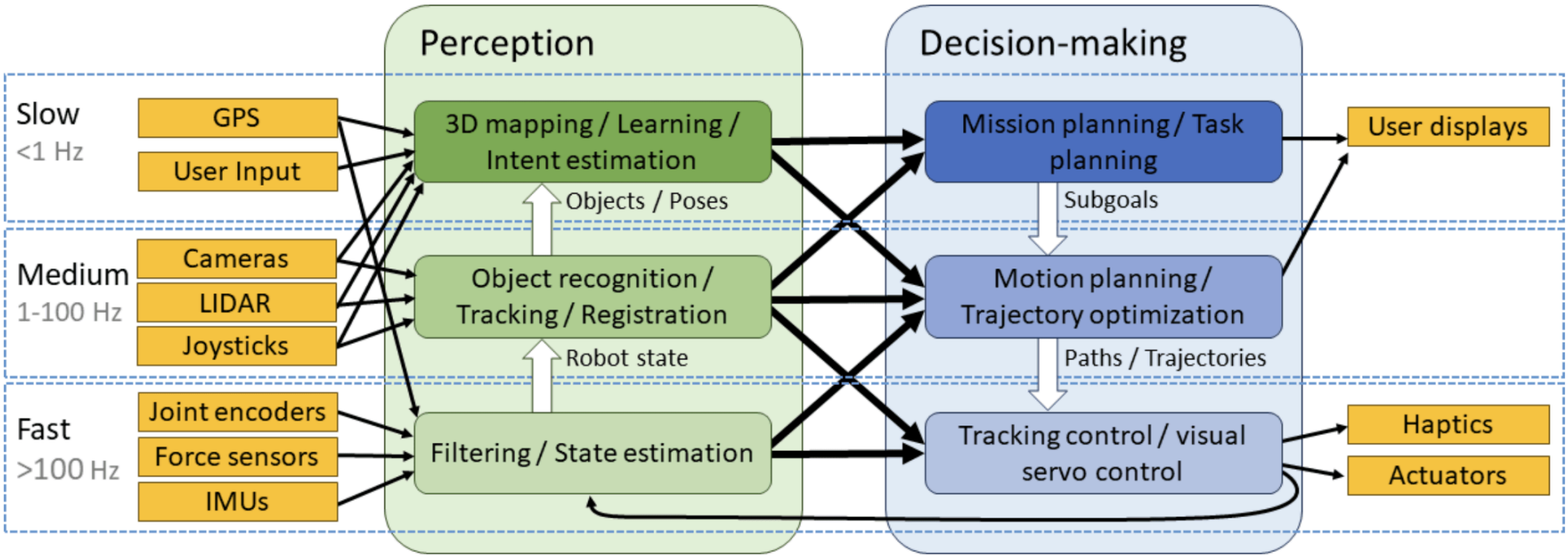




Low-level feedback control from cameras / sensors

(continuous time/state/action)

Robot System Architecture

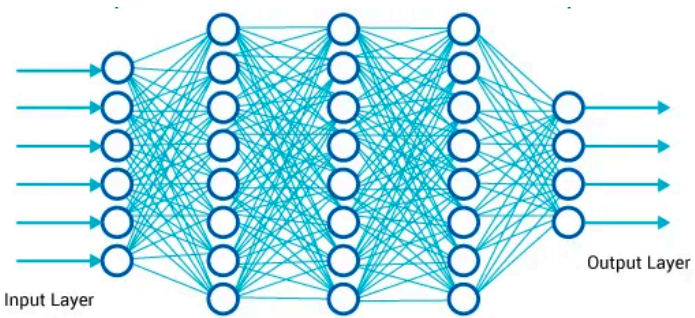


From Kris Hauser - Robotic Systems (online)

Visuomotor policies

perception network

(often pre-trained)



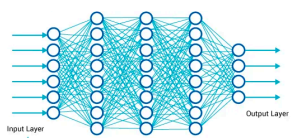
$\mathbb{R}^{640\times480\times3}$

$\approx \mathbb{R}^{32}$

z

learned state representation

policy network



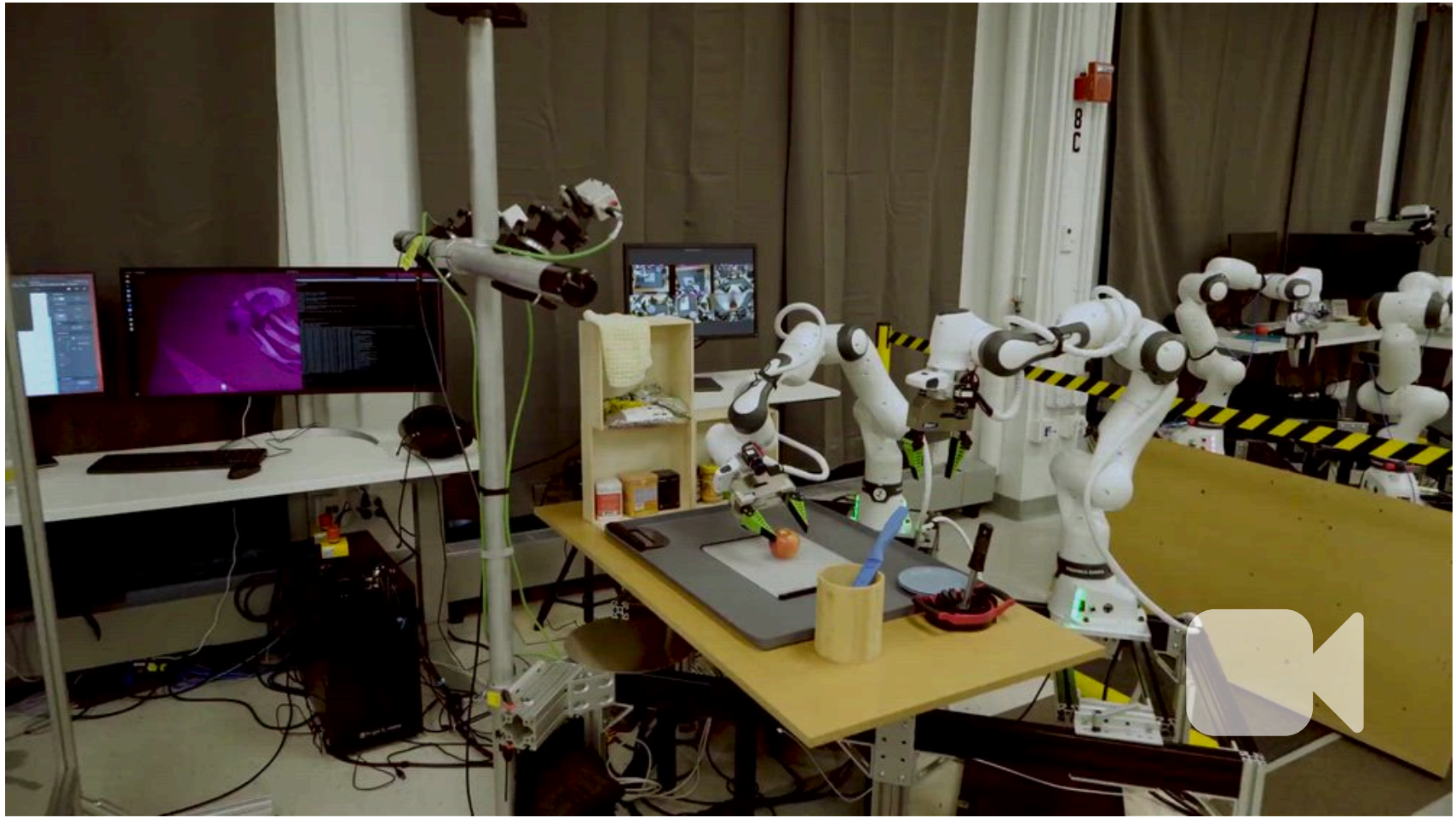
$\approx \mathbb{R}^{7}$

actions

other robot sensors

$\approx \mathbb{R}^{13}$

Levine*, Finn*, Darrel, Abbeel, JMLR 2016



Manipulation is much more than "pick and place"!

TOYOTA RESEARCH INSTITUTE

https://www.youtube.com/embed/HYwekersccY?enablejsapi=1

But even pick and place is challenging in the general case



C. Garrett, C. Paxton, T. Lozano-Perez, L. Kaelbling, D. Fox 2020

What is Drake?



CORE LIBRARY

Modeling Dynamical Systems	Solving Mathematical Programs	Multibody Kinematics and Dynamics
API | TUTORIAL	API | TUTORIAL	API

drake.mit.edu

Drake features

- Systems framework
- Deterministic simulations
- Optimization framework
- Sophisticated contact modeling (hydroelastic)

Signals and Systems Abstraction

Denny Freeman

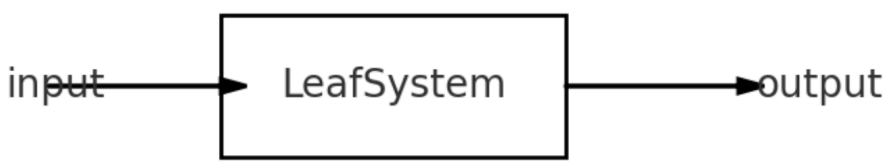
https://ocw.mit.edu/courses/6-01sc-introduction-to-electrical-engineering-and-computer-science-i-spring-2011/9f07e459d9366a4a1b7deda810cf0345_MIT6_01SCS11_lec03.pdf

State

Systems Framework

Drake models computations as Systems, wired together in a Diagram.

- LeafSystem: user-defined input/output and dynamics.
- Diagram: composition of multiple subsystems.
- Context: stores state, time, parameters, inputs.
- Ports: inputs and outputs that connect systems.



Example: Simple LeafSystem

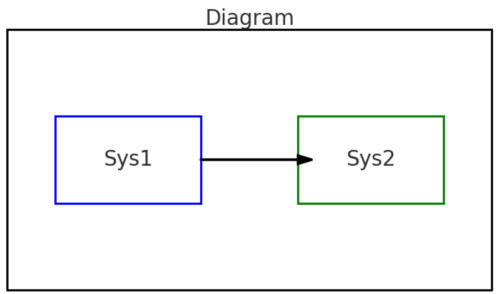
```
class DoubleSystem(LeafSystem):
    def __init__(self):
        super().__init__()
        self.DeclareVectorInputPort('u', 1)
        self.DeclareVectorOutputPort('y', 1, self.CalcY)

    def CalcY(self, context: Context, output: BasicVector) -> None:
        u = self.get_input_port('u').Eval(context)
        output.SetFromVector([2*u[0]])
```

- a = BasicVector(3) # 3-d vector
- a.SetFromVector(array) # from numpy array
- a.CopyToVector() # to numpy array

Diagram Composition

- Diagrams allow composition of multiple subsystems.
  - Each system has input/output ports.
  - Ports are connected to build larger systems.




```
builder = DiagramBuilder()
builder.AddSystem(sys1)
builder.AddSystem(sys2)
builder.Connect(sys1.get_output_port(0), sys2.get_input_port(0))
diagram = builder.Build()
```

Dynamic systems

```python
class VerticalMassUnderGravity(LeafSystem):
    """
    Continuous-time 1D vertical mass:
        ż = v
        v̇ = g + f/m   (constant gravity)
    State vector: x = [z, v]
    """
    def __init__(self, gravity=-9.81, mass=1.0):
        super().__init__()
        self._g = float(gravity)
        self._m = float(mass)

        # Continuous state: q (z), v (ż)
        self.DeclareContinuousState(2)
        # Input force f
        self.DeclareVectorInputPort("force", 1)

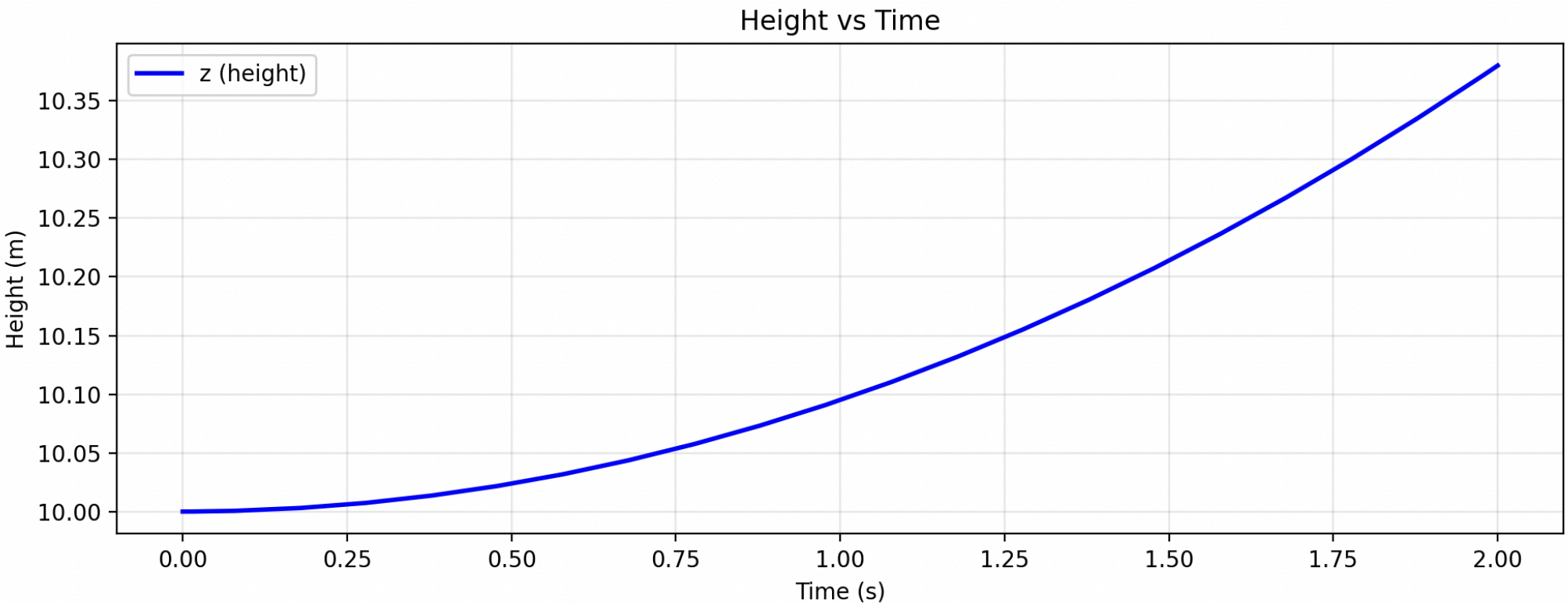
        # Output: position
        self.DeclareVectorOutputPort("position", 1, self.CalcPosition)

    def CalcPosition(self, context: Context, output: BasicVector) -> None:
        x = context.get_continuous_state_vector()
        output.SetFromVector([x[0]])

    def DoCalcTimeDerivatives(self, context:Context, derivatives:ContinuousState) -> None:
        x = context.get_continuous_state_vector().CopyToVector()
```

Simulation

```
1  if __name__ == "__main__":
2
3      builder = DiagramBuilder()
4      mass_sys = builder.AddSystem(VerticalMassUnderGravity())
5
6      # Two ways of doing the logger
7      logger = LogVectorOutput(mass_sys.get_output_port(0), builder)
8      # logger = builder.AddSystem(VectorLogSink(1))
9      # builder.Connect(mass_sys.get_output_port(0), logger.get_input_port(0))
10
11     diagram = builder.Build()
12
13     ctx = diagram.CreateDefaultContext()
14     sim = Simulator(diagram, ctx)
15
16     ctx.get_mutable_continuous_state().SetFromVector([10.0, 0.0])
17     sys_ctx = mass_sys.GetMyContextFromRoot(ctx)
18     mass_sys.get_input_port(0).FixValue(sys_ctx, 10.0)
19
20     sim.AdvanceTo(2.0)  # simulate 2 seconds
21     z_final = mass_sys.get_output_port(0).Eval(sys_ctx)
22
23     print("Final position [z] =", z_final)
24
25     plot(sim)
```


Height vs Time
z (height)
Height (m)
10.35
10.30
10.25
10.20
10.15
10.10
10.05
10.00
0.00
0.25
0.50
0.75
1.00
1.25
1.50
1.75
2.00
Time (s)

Drake's Physics Engine: MultibodyPlant

MultibodyPlant is Drake's class for modeling robots and mechanisms.

- Add bodies, joints, actuators, forces.
- Register geometry with SceneGraph for visualization/collision.
- Parse models from URDF/SDF files.
- Requires Finalize() after model is built to produce System

```
builder = DiagramBuilder()
plant, scene_graph = AddMultibodyPlantSceneGraph(builder, time_step=1e-4)
parser = Parser(plant, scene_graph)
parser.AddModelsFromUrl("package://drake_models/iiwa_description/urdf/iiwa14_primitive_c
plant.Finalize()
diagram = builder.Build()
```

https://people.csail.mit.edu/russt/uploads/yellow_spot.html

Contact simulation





https://manipulation.csail.mit.edu/Fall2026/schedule.html